

Lecture 09: Introduction to the DOM (Document Object Model)

Static vs. Dynamic Web Pages:

- **The Problem:** Pure HTML creates static websites. If you want to change content, you must manually edit the source code.
- **The Solution:** JavaScript introduces interactivity, allowing the page to dynamically change (e.g., updating text immediately upon a button click) without manual edits.

HTML Elements as JavaScript Objects:

In JavaScript, almost everything (including arrays and functions) operates as an Object. To allow JavaScript to communicate with an HTML document, the browser automatically converts standard HTML elements into structured JavaScript Objects.

HTML Code:

```
<h1 id="first" style="background-color: lightsteelblue; color: midnightblue; font-size: 50px;">
Hello Thunder Students
</h1>
```

Internal JavaScript Object Representation:

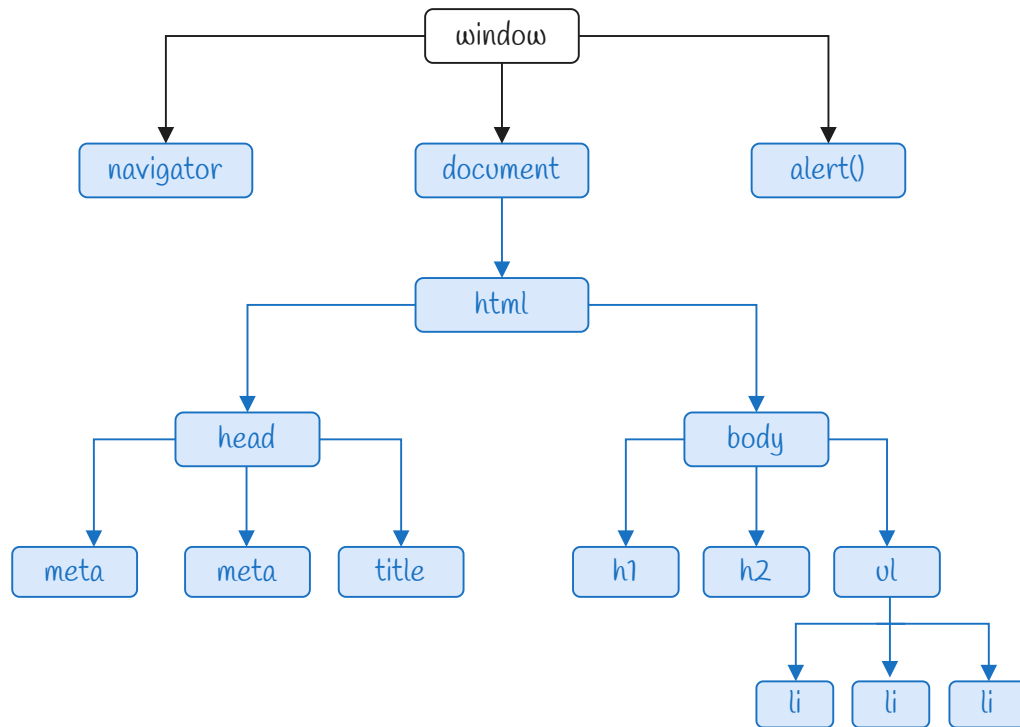
```
const element = {
  tagName: "h1",
  id: "first",
  textContent: "Hello Thunder Students",
  style: {
    backgroundColor: "pink",
    color: "black",
    fontSize: "50px"
  }
};
```

The Superpower: Once the browser converts the HTML into an object, you can dynamically manipulate its properties directly via JavaScript (e.g., `element.textContent = "Hello Coder Army"`). The browser's UI will immediately reflect this change.

The window and document Objects:

- **The window Object:** This is the massive global object created by the browser. It contains pre-built methods and properties like `alert()`, `location`, and even `console.log()`. Because it is global, properties can be accessed directly without explicitly typing `window.console.log()`.
- **The document Object:** The browser takes the entire loaded HTML page, parses it, and stores it as an object named `document` inside the window object (`window.document`). This structure can be inspected by executing `console.dir(document)`; in the console.

DOM Hierarchy Visualized:



Selecting DOM Elements:

To modify an element, it must first be selected (targeted) from the document.

Method	Return Type	Description
<code>getElementById("idName")</code>	Single Object	Targets a unique element by its ID and returns a single object.
<code>getElementsByClassName("className")</code>	HTMLCollection	Selects all elements sharing a specific class name.
<code>getElementsByTagName("tagName")</code>	HTMLCollection	Selects all elements of a specific tag.
<code>getElementsByName("nameValue")</code>	NodeList	Targets elements by their name attribute. (Highly useful for grabbing form input fields or radio buttons).
<code>querySelector("selector")</code>	Single Object	The modern standard. Returns the first element that matches a specific CSS selector.
<code>querySelectorAll("selector")</code>	NodeList	Returns all elements that match a given CSS selector in the form of a static NodeList.

```
const element = document.getElementById('first');
element.textContent = "Hello Coder Army";
```

Array-Like Objects (HTMLCollection & NodeList): Methods targeting classes or tags return an HTMLCollection, while queries return a NodeList. Both are array-like lists, but they are not exact arrays, meaning standard array methods (like `.map()` or `.filter()`) cannot be used directly on them without converting them first.

Text Manipulation Methods:

There are three primary properties used to extract or inject text into an HTML element.

Property	Description	Usage Note
<code>.innerText</code>	Displays only the text that is strictly visible on the screen (ignores hidden text).	Safe
<code>.textContent</code>	Extracts the pure text content of the element, including hidden parts.	Best Practice
<code>.innerHTML</code>	Considers everything passed to it as literal HTML and renders the tags.	Dangerous

The Security Risk of .innerHTML (XSS):

- If user-generated data (like a comment section) is injected using .innerHTML, an attacker can inject malicious code.
- **Example:** ``
- Because .innerHTML executes this as code, a malicious script can read cookies or steal sensitive data.
- **The Fix:** Always use .textContent for user inputs so the browser treats the input strictly as a harmless string, rather than executable code. Modern browsers are highly secure and restrict scripts from accessing data outside the current website.

Modifying Styles, Attributes and Classes:

Once an element is selected, its structural attributes, CSS classes, and inline styles can be dynamically manipulated.

The .style Property:

The .style property allows direct modification of an element's inline CSS. CSS properties with hyphens are converted to camelCase in JavaScript.

```
const element = document.getElementById('first');
```

```
element.style.backgroundColor = "skyblue";  
element.style.fontSize = "60px";
```

Attributes and Classes:

Elements possess various attributes (like id, src, or href) and classes that dictate their behavior and styling. These can be accessed and reassigned directly as properties of the DOM object. The .classList property provides built-in methods to safely add, remove, or toggle classes without overriding the entire class string.

```
const element = document.getElementsByClassName("classFirst");
```

```
element[1].id = "second";  
element[1].classList.add("classSecond");
```

How Does a Browser Refresh Work?

When a browser refreshes, it deallocates the old memory stored in the RAM. It then fetches the latest code and allocates fresh memory to load the new page.